

Mobile Application

UNIT 1(21 makrs)

CHAPTER 1: Programming Revision (Object Oriented Programming Concepts & Java Fundamental): (12 marks)

Class, Object & methods, Constructors in Java, Inheritance, Polymorphism, Abstraction, Encapsulation, Exception Handling in Java, Multithreading in Java, File I/O

Introduction to Android Programming Language: What is Android, History and Version, Software Stack, Core Building Blocks, Android Emulator, Hello Android example, Internal Details, Dalvik VM, AndroidManifest.xml

CHAPTER 2: Android Application Layout:(9 marks)

Android Linear Layout, Android Relative Layout, Android Table Layout, Scroll View in Android, Android Frame Layout

UNIT 2(33 marks)

CHAPTER 3: Android Activity & Intent, Android Fragments: (9 makrs)

Activity Lifecycle, Implicit Intent, Explicit Intent, Android Fragments

CHAPTER 4: Android UI widgets:(18 marks)

Working with Button, Toast, Toggle Button, Checkbox, Image View, Image Button, Alert Dialog, Spinner, AutoComplete Text View, Rating Bar, Date Picker, Time Picker, Progress Bar

CHAPTER 5: Building Android Application with Web View: (6 makrs)

Building Simple Web View Application, Load, HTML Data on Web View, Embed/Display YouTube Video in Web View, Convert Custom Design Website into Android App.

UNIT 3(12 makrs)

CHAPTER 6: Menus in Android & Services:(3 marks)

Option Menu, Context Menu, Popup Menu, It can be used to perform any task in background. It doesn't have any user interface (UI).

CAHPTER 7: Android Database (SQLite) & Android Multimedia:(3 marks)

SQLite Example with GUI, Fire Base (Real time), Playing Audio in android Example, Playing Video in android Example, Playing Media Player in android Example.

CHAPTER 8: Android Speech & Telephony API:(6 marks)

Speech API is used to convert text into speech, Text to Speech Example with Speed option, Telephony Manager, Get Call State, Call State Broadcast Receiver, How to make a Phone Call, How to Send SMS, How to Send Email.

UNIT 4(9 marks)

CAHPTER 9: Device Connectivity &Android sensor: (3 marks)

Bluetooth, List Paired Device, Wi-Fi, Android Sensor.

CHAPTER 10: Android Material Design Using Design Support Library & Animation: (6 marks)

Navigation Drawer View, Splash Screen, Android animation enables you to rotate, slide and flip images and text, Fade In Animation in Android, Fade Out Animation in Android, Zoom In Animation in Android, Zoom Out Animation in Android, Implementing Ripple Effect in Android, Add Ripple Effect/Animation to a Android Button.

UNIT 1

1. What is OOPS? Explain OOPS concept with JAVA .

ANS:- Introduction

Object-Oriented Programming System (OOPS) is a programming paradigm that organizes software design around objects rather than functions or procedures. Unlike traditional procedural programming, which focuses on writing sequences of instructions to manipulate data, OOPS emphasizes creating reusable objects that encapsulate both data (attributes) and behavior (methods). This approach models real-world entities more naturally, making software easier to develop, maintain, and extend. Java is a pure object-oriented language, as almost everything in Java is treated as an object, and its architecture strongly supports OOPS principles. OOPS provides advantages such as modularity, reusability, flexibility, and security, which are essential in modern software development.

OOPS in Java

In Java, OOPS is implemented through classes and objects. A class acts as a blueprint defining the properties (fields) and behaviors (methods) of objects. When an object is created from a class, it is called an instance of that class and can access its fields and methods. This encapsulation of data and behavior makes Java programs organized and manageable. Java provides access modifiers like private, public, and protected to control visibility and enhance data security. The use of packages and class hierarchies also supports OOPS design, making large applications modular and easier to maintain.

1.Encapsulation

Encapsulation is the principle of combining data and methods into a single unit called a class, while restricting direct access to some of the object's components. In Java, this is implemented using private fields and public getter/setter methods. Encapsulation ensures that the internal state of an object cannot be modified arbitrarily from outside the class, improving data security and maintainability.

Example:

```
class Student {  
    private String name;  
    private int age;  
    public String getName() {  
        return name;  
    }  
    public void setName(String name) {  
        this.name = name;  
    }  
    public int getAge() {  
        return age;  
    }  
    public void setAge(int age) {
```

```
        this.age = age;
    }
}
```

Here, the fields name and age are **private**, and they can only be accessed or modified through getter and setter methods.

2.Inheritance

Inheritance allows one class (the child class) to acquire the properties and methods of another class (the parent class). Java uses the keyword `extends` to implement inheritance. This promotes code reusability and represents the “is-a” relationship in real life. A child class can also override methods of the parent class to provide its own implementation.

Example:

```
class Animal {
    void eat() {
        System.out.println("Animal eats");
    }
}
```

```
class Dog extends Animal {
    void eat() {
        System.out.println("Dog eats");
    }
}
```

Here, Dog inherits from Animal and overrides the `eat()` method to provide a specific behavior.

3.Polymorphism

Polymorphism means "many forms." In Java, polymorphism allows a single method or object to behave differently in different contexts. There are two types:

- Compile-time Polymorphism (Method Overloading): Multiple methods have the same name but different parameters.
- Runtime Polymorphism (Method Overriding): The method executed depends on the actual object at runtime.

Example of Method Overloading:

```
class Calculator {
    int add(int a, int b) {
        return a + b;
    }
    double add(double a, double b) {
        return a + b;
    }
}
```

4.Abstraction

Abstraction focuses on hiding unnecessary details while showing only the essential features. In Java, abstraction is implemented using abstract classes and interfaces. Abstract classes may contain both abstract methods (without body) and concrete methods (with body). Interfaces contain only abstract methods, which must be implemented by classes.

Example:

```
abstract class Vehicle {  
    abstract void start();  
}  
  
class Car extends Vehicle {  
    void start() {  
        System.out.println("Car starts with key");  
    }  
}
```

Here, Vehicle defines the essential behavior, but the implementation is provided in the subclass Car.

Additional OOP Features in Java

In addition to the main principles, Java supports:

- **Constructors:** Initialize objects.
- **Method Overloading and Overriding:** Supports polymorphism.
- **Interfaces and Packages:** Provides modularity.
- **Exception Handling:** Enhances reliability.
- **Garbage Collection:** Automatic memory management complements OOPS by handling object lifecycle efficiently.

2. Define Android. Explain in detail Android Architecture with a neat and clean diagram.**ANS:- Introduction**

Android is an open-source, Linux-based operating system designed primarily for mobile devices such as smartphones and tablets. It combines an operating system, middleware, and core applications, providing a complete platform for developing and running mobile applications. Android is developed and maintained by Google, and its open-source nature allows developers to customize and extend the system according to their requirements. The platform supports multitasking, connectivity, rich multimedia, and security, making it one of the most widely used operating systems for mobile computing. Understanding Android architecture is essential for developers as it explains how applications interact with the underlying system, ensuring efficiency, modularity, and performance.

Android Architecture

Android architecture is a layered design that separates the hardware, system services, and applications, providing a modular and flexible environment. The architecture can be broadly divided into Linux Kernel, Hardware Abstraction Layer, Android Runtime, Libraries, Application Framework, and Applications. Each layer serves a specific purpose, and together they provide a robust and efficient platform for application development.

1. Linux Kernel

At the foundation of Android lies the Linux Kernel, which manages core system services such as process management, memory management, networking, and security. It also provides drivers for essential hardware components like display, camera, audio, and sensors. By using the Linux kernel, Android ensures hardware abstraction, stability, and security, allowing applications to run consistently across different devices.

2.Hardware Abstraction Layer (HAL)

The Hardware Abstraction Layer (HAL) provides a standard interface for communicating with device hardware. HAL allows system services and applications to access hardware components like cameras, GPS, and Bluetooth without needing to understand the low-level implementation details. This abstraction ensures hardware compatibility and simplifies development.

3.Android Runtime (ART) and Core Libraries

The Android Runtime (ART) is responsible for executing Android applications. It includes a virtual machine that converts Java or Kotlin bytecode into instructions executable by the device. ART provides just-in-time (JIT) compilation and ahead-of-time (AOT) compilation, improving performance and efficiency. The runtime also provides core Java libraries for data structures, file handling, network communication, and graphics, allowing applications to use standard functionalities easily.

4.Libraries

Android provides a set of C/C++ libraries to support various features:

- **Surface Manager:** Handles display and graphics rendering.
- **Media Libraries:** Supports audio, video, and image formats.
- **SQLite:** Lightweight database for storage.
- **WebKit:** Enables web browsing features.
- **OpenGL ES:** Provides 2D and 3D graphics rendering.

These libraries form an intermediate layer between the runtime and application framework, offering high-level APIs for application developers.

5.Application Framework

The Application Framework provides high-level APIs to simplify application development. Key components include:

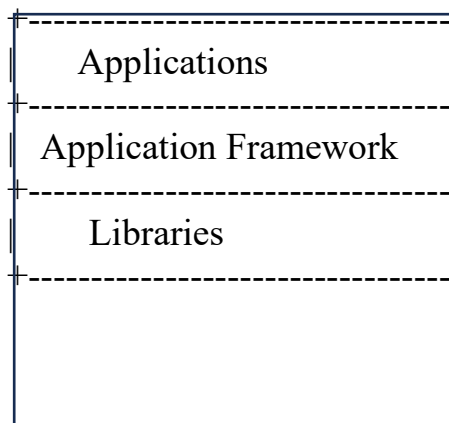
- **Activity Manager:** Manages application lifecycle.
- **Content Providers:** Facilitate sharing data between apps.
- **Resource Manager:** Handles strings, layouts, and graphics.
- **Notification Manager:** Displays system notifications.
- **Location Manager:** Provides location-based services.

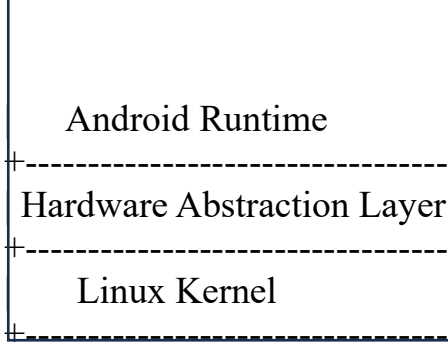
By using the application framework, developers can focus on building application features without worrying about hardware or low-level system details.

6.Applications

At the top layer are the Applications. These include both pre-installed system apps like Phone, Messaging, and Browser, as well as user-installed third-party applications. Applications interact with the system through the application framework and ultimately through the runtime and libraries, ensuring seamless operation across devices.

Android Architecture Diagram





3. What is Exception? What are the types of Exception? How Exception can be handle in JAVA?

ANS:- Introduction

In Java, an exception is an event that occurs during program execution, disrupting its normal flow. Exceptions arise due to runtime errors such as invalid input, file not found, division by zero, or network failure. Handling exceptions is critical for creating robust and fault-tolerant programs. Java provides a powerful exception handling mechanism, enabling programs to detect and respond to errors without crashing. Proper understanding of exceptions and their types is essential for developing reliable applications.

What is an Exception

An exception is an object representing an abnormal condition during program execution. When an exception occurs, Java creates and throws an exception object, which can then be caught and handled. Unlike errors like `OutOfMemoryError`, exceptions are generally recoverable. Effective exception handling ensures the program continues execution or exits gracefully while providing meaningful feedback to the user.

Types of Exceptions

Java categorizes exceptions into two types:

Checked Exceptions

Checked exceptions are checked at compile time. They occur due to external conditions outside the program, such as file I/O or database errors. If not handled with try-catch or declared using throws, the program will fail to compile.

- **IOException** – input/output failure
- **SQLException** – database access error
- **FileNotFoundException** – file does not exist

Unchecked Exceptions

Unchecked exceptions occur at runtime and are not checked by the compiler. They are usually caused by programming errors like accessing invalid indexes or null references.

- **ArithmeticException** – division by zero
- **NullPointerException** – accessing null objects
- **ArrayIndexOutOfBoundsException** – invalid array access

Exception Handling in Java

Java provides five keywords for handling exceptions: try, catch, finally, throw, and throws. These allow programmers to catch runtime errors and provide controlled responses.

Try and Catch

The try block contains code that may throw an exception, while the catch block handles it.

Example:

```
try {  
    int result = 10 / 0;  
} catch (ArithmeticException e) {  
    System.out.println("Cannot divide by zero: " + e.getMessage());  
}  
System.out.println("Program continues...");
```

Finally Block

The finally block executes regardless of exceptions, commonly used to release resources.

Example:

```
try {  
    // code  
} catch (Exception e) {  
    // handle exception  
} finally {  
    System.out.println("Always executes");  
}
```

Throw and Throws

- throw is used to explicitly throw an exception.
- throws is used to declare exceptions in a method signature.

Example:

```
void divide(int a, int b) throws ArithmeticException {  
    if(b == 0)  
        throw new ArithmeticException("Cannot divide by zero");  
    else  
        System.out.println(a / b);  
}
```

4. What is Thread in Java and Discuss the Life Cycle of a Thread

ANS:- Introduction

In Java, a thread is a lightweight subprocess that allows a program to perform multiple tasks concurrently. Each thread represents a separate path of execution within a program, sharing the same memory space with other threads of the process. Multithreading improves performance and resource utilization, especially for tasks that are independent or require parallel execution. Java provides built-in support for multithreading through the Thread class and the Runnable interface, making it easier for developers to write concurrent and efficient applications.

What is a Thread

A thread is an independent flow of control in a program. Unlike a single-threaded application, which executes one instruction at a time, multithreaded programs can execute multiple threads simultaneously, resulting in faster execution and better responsiveness. In Java, a thread can be created either by:

1. **Extending the Thread class** and overriding its run() method.
2. **Implementing the Runnable interface** and defining the run() method.

Example using Thread class:

```
class MyThread extends Thread {
    public void run() {
        System.out.println("Thread is running");
    }
}

public class TestThread {
    public static void main(String[] args) {
        MyThread t1 = new MyThread();
        t1.start(); // starts a new thread
    }
}
```

Example using Runnable interface:

```
class MyRunnable implements Runnable {
    public void run() {
        System.out.println("Runnable thread running");
    }
}

public class TestRunnable {
    public static void main(String[] args) {
        Thread t1 = new Thread(new MyRunnable());
        t1.start();
    }
}
```

Life Cycle of a Thread

A thread in Java goes through **five distinct states**, collectively known as the **thread life cycle**. Understanding this life cycle is crucial for proper thread management and synchronization.

1. New (Created) State

When a thread is instantiated using the Thread class or Runnable interface, it is in the new state. In this state, the thread is not yet running. The thread moves to the Runnable state only when the start() method is invoked.

2. Runnable State

After calling `start()`, the thread enters the Runnable state, meaning it is ready to run but waiting for the CPU scheduler to allocate processor time. Threads in this state compete for CPU time, and the exact order of execution is determined by the thread scheduler.

3. Running State

When the thread scheduler selects a thread from the Runnable pool, the thread moves to the Running state, where its `run()` method executes. A thread remains in this state until it completes its task, is blocked, or is preempted by a higher-priority thread.

4. Waiting/Blocked State

A thread can enter the waiting or blocked state when it waits for another thread to perform a particular action or when it is waiting for resources. A thread in this state does not consume CPU time. It remains blocked until it is notified or the resource becomes available.

5. Terminated (Dead) State

Once the thread completes execution or is stopped, it enters the terminated state. A terminated thread cannot be restarted by calling `start()` again. To execute the task again, a new thread instance must be created.

5. Discuss Core Building Blocks of Android.

ANS:- Introduction

Android applications are built using **a set of core components** that define the structure, functionality, and behavior of an app. These **building blocks** provide a standard way to create modular, reusable, and interactive applications. Understanding these components is essential for developing efficient Android apps that interact seamlessly with the system and user interface.

Activities

An Activity represents a single screen with a user interface in an Android application. Each activity is responsible for handling user interactions, displaying content, and responding to events. Activities follow a lifecycle, including states like `onCreate()`, `onStart()`, `onResume()`, `onPause()`, `onStop()`, and `onDestroy()`, which allow developers to manage resources efficiently.

Example: A login screen or a settings page can be implemented as separate activities.

Services

A Service is a component that performs long-running operations in the background without a user interface. Services are used for tasks such as playing music, downloading files, or fetching data from the internet. Services can run in the same process as the application or in a separate process, and they can be started or bound depending on the requirements.

Example: A music player continues to play music even when the user switches to another app.

Broadcast Receivers

Broadcast Receivers allow applications to listen for system-wide or application-wide broadcasts and respond accordingly. They enable apps to react to events such as incoming calls, low

battery, or network changes. Broadcast receivers can start an activity, service, or perform a specific action in response to the broadcast.

Example: An app can notify the user when the device battery is low.

Content Providers

Content Providers manage shared data between applications. They provide a standard interface to access, insert, update, or delete data from sources such as SQLite databases, files, or web services. Content providers enable data encapsulation and controlled access, ensuring security and consistency.

Example: The Contacts app uses a content provider to allow other apps to read contact information safely.

Intent

An Intent is a messaging object that allows communication between components in an Android application. Intents are used to start activities, services, or send broadcasts. There are two types: Explicit Intents (specifying the target component) and Implicit Intents (allowing the system to choose the appropriate component).

Example: Opening a webpage from an app can be done using an implicit intent.

Fragments

A Fragment represents a modular section of an activity. Fragments allow developers to create flexible and reusable UI components that can be combined in different activities. They have their own lifecycle methods similar to activities, such as `onCreateView()` and `onDestroyView()`, and are ideal for dynamic UI changes.

Example: A news app may display a list of articles in one fragment and the article content in another.

6. What is Layout? Explain Different types of Layouts in Android (Linear, Relative, Table, Frame).

ANS:- Introduction

In Android, a layout is a structure or container that defines how user interface (UI) elements such as buttons, text views, images, and other widgets are arranged on the screen. Layouts determine the position, size, and alignment of these elements, ensuring that the app interface is organized, visually appealing, and responsive on different devices. Understanding various types of layouts helps developers design flexible and efficient UI designs suitable for different screen sizes and orientations.

Linear Layout

A Linear Layout arranges UI elements in a single direction, either vertically or horizontally. It is simple and widely used when elements need to be displayed in a sequential order. Developers can use the orientation attribute to specify the layout direction. Weight and gravity attributes can also control space distribution and alignment within the layout.

Example:

```
<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical">
    <Button
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Button 1"/>
    <Button
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Button 2"/>
</LinearLayout>
```

Relative Layout

A Relative Layout allows UI elements to be positioned relative to each other or to the parent container. It provides flexibility for complex interfaces by using attributes like `layout_alignParentTop`, `layout_below`, `layout_toRightOf`, etc. This layout is useful when elements need to adjust dynamically based on other components.

Example:

```
<RelativeLayout
    android:layout_width="match_parent"
    android:layout_height="match_parent">
    <TextView
        android:id="@+id/text1"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Hello"/>
    <Button
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Click"
        android:layout_below="@id/text1"/>
</RelativeLayout>
```

Table Layout

A Table Layout organizes UI elements in a grid of rows and columns, similar to an HTML table. Each row is defined by a `TableRow` element, and child views within the row represent columns.

Table layouts are ideal for displaying structured data or creating forms with multiple aligned input fields.

Example:

```
<TableLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content">
    <TableRow>
        <TextView android:text="Name"/>
        <EditText android:layout_width="wrap_content"/>
    </TableRow>
    <TableRow>
        <TextView android:text="Email"/>
        <EditText android:layout_width="wrap_content"/>
    </TableRow>
</TableLayout>
```

Frame Layout

A Frame Layout is designed to hold a single child view, or multiple views stacked on top of each other. It is commonly used to display a single widget or overlay views. Frame layouts are simple but effective when layering UI elements or managing dynamic content.

Example:

```
<FrameLayout
    android:layout_width="match_parent"
    android:layout_height="match_parent">
    <ImageView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:src="@drawable/background"/>
    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Overlay Text"/>
</FrameLayout>
```

7. What is Relative Layout? Explain any six attributes of Relative Layout.

ANS:- Introduction

A Relative Layout in Android is a view group that arranges its child elements relative to each other or to the parent container. It provides flexibility in positioning UI components, allowing developers to align, position, or anchor views dynamically. Relative Layout is particularly useful when designing complex interfaces, as it reduces the need for nested layouts and ensures

better performance. By using relative positioning, UI elements can adapt to different screen sizes and orientations.

Six Important Attributes of Relative Layout

1. layout_alignParentTop

This attribute positions a child view aligned with the top edge of the parent container. It is commonly used when you want an element to appear at the top of the screen.

Example:

```
<Button
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Top Button"
    android:layout_alignParentTop="true"/>
```

2. layout_alignParentBottom

This attribute aligns a child view with the bottom edge of the parent container. It ensures that the view remains anchored at the bottom, regardless of other components.

Example:

```
<Button
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Bottom Button"
    android:layout_alignParentBottom="true"/>
```

3. layout_centerInParent

The layout_centerInParent attribute centers a view both horizontally and vertically within the parent container. It is useful for elements like logos or main buttons that need central positioning.

Example:

```
<TextView
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Center Text"
    android:layout_centerInParent="true"/>
```

4. layout_toRightOf / layout_toLeftOf

These attributes position a view relative to another sibling view.

- layout_toRightOf places a view to the right of another view.
- layout_toLeftOf places a view to the left of another view.

Example:

```
<TextView
    android:id="@+id/text1"
```

```
    android:text="Label"/>
<Button
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Click"
    android:layout_toRightOf="@id/text1"/>
```

5. layout_above / layout_below

These attributes position a view above or below another view in the layout. They are useful for stacking elements vertically while maintaining relative positions.

Example:

```
<TextView
    android:id="@+id/text2"
    android:text="Text 2"/>
<Button
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Submit"
    android:layout_below="@id/text2"/>
```

6. layout_alignParentLeft / layout_alignParentRight

These attributes align a view to the left or right edge of the parent container. They are commonly used for navigation buttons or side-aligned elements.

Example:

```
<Button
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Left Button"
    android:layout_alignParentLeft="true"/>
```

UNIT 2

1. What is Intent? Explain its types (Explicit and Implicit) with necessary attributes.

ANS:- Introduction

In Android, an Intent is a messaging object used to communicate between components such as activities, services, and broadcast receivers. Intents are essential for starting new activities, sending data, or triggering system actions. They provide a flexible mechanism for inter-component communication, enabling applications to interact seamlessly within the system or with other apps. Intents can be categorized into Explicit Intents and Implicit Intents, each serving a specific purpose in Android app development.

an Intent is a key component in Android for communication between app components. It allows applications to start activities, services, or broadcast messages efficiently. There are two main types: Explicit Intents, which target a specific component, and Implicit Intents, which let the system select the appropriate component based on the action and data. By understanding intents and their attributes—such as component, action, data, category, extras, and flags—developers can create flexible, interactive, and dynamic Android applications.

Explicit Intent

An Explicit Intent is used when the target component is specifically defined. It is commonly used to start a particular activity or service within the same application. Developers explicitly specify the class name or component name in the intent object, ensuring that the action is delivered to the intended target.

Example:

```
Intent intent = new Intent(MainActivity.this, SecondActivity.class);
startActivity(intent);
```

Here, the intent explicitly specifies that SecondActivity should be launched from MainActivity.

Necessary Attributes of Explicit Intent:

- **Component Name:** Specifies the **target activity or service**.
- **Data:** Optional extra data to be passed to the target.
- **Flags:** Optional instructions about how the activity should be launched (e.g., FLAG_ACTIVITY_NEW_TASK).

Explicit intents are **simple and reliable**, as the target component is always known.

Implicit Intent

An Implicit Intent does not specify a particular component. Instead, it declares an action to be performed, allowing the system to determine the best-suited component to handle the request. Implicit intents are commonly used for interacting with other apps, such as opening a web page, dialing a number, or sending an email.

Example:

```
Intent intent = new Intent(Intent.ACTION_VIEW, Uri.parse("https://www.example.com"));
startActivity(intent);
```

Here, the system selects an app (like a web browser) capable of handling the ACTION_VIEW action.

Necessary Attributes of Implicit Intent:

- **Action:** Defines the **general operation** to perform, such as ACTION_VIEW or ACTION_SEND.
- **Data:** Provides a **URI or additional information** for the action.
- **Category:** Optional, further defines the **type of component** that can handle the intent.
- **Extras:** Optional additional key-value data sent to the receiving component.
- **Flags:** Optional instructions about **how the intent should be handled**.

Implicit intents allow Android apps to **interact with external applications** without requiring explicit knowledge of the target component.

2. What is Activity? Explain Activity Life Cycle with a diagram.

ANS:- Introduction

An Activity is a single screen with a user interface that allows users to interact with the application. Each activity represents a distinct part of the app, such as a login screen, settings page, or dashboard. Activities are central to Android app development because they manage UI elements, respond to user inputs, and handle navigation between different screens.

Understanding the activity life cycle is essential for developers to manage resources, save state, and ensure smooth user experience.

What is Activity

An Activity is a component of an Android application that serves as the entry point for user interaction. It provides a window where UI elements like buttons, text fields, and images are displayed, and it handles events such as clicks, touches, or gestures. Activities can interact with each other using Intents, enabling seamless navigation and data sharing between screens.

Example:

```
public class MainActivity extends AppCompatActivity {  
    @Override  
    protected void onCreate(Bundle savedInstanceState) {  
        super.onCreate(savedInstanceState);  
        setContentView(R.layout.activity_main);  
    }  
}
```

Here, MainActivity represents an activity with its layout defined in activity_main.xml.

Activity Life Cycle

The Activity Life Cycle describes the stages an activity goes through from creation to destruction. Managing these states properly is crucial for efficient resource usage, data persistence, and user experience. The main life cycle methods include:

1. onCreate()

Called when the activity is first created. This is where you initialize components, set the layout, and bind data.

2. onStart()

Invoked when the activity becomes visible to the user but is not yet interactive.

3. onResume()

Called when the activity starts interacting with the user. The activity is now in the foreground and responsive.

4. onPause()

Called when the system is about to move the activity to the background. This is used to save data or stop animations.

5. onStop()

Invoked when the activity is no longer visible. Resources like sensors or background tasks should be released here.

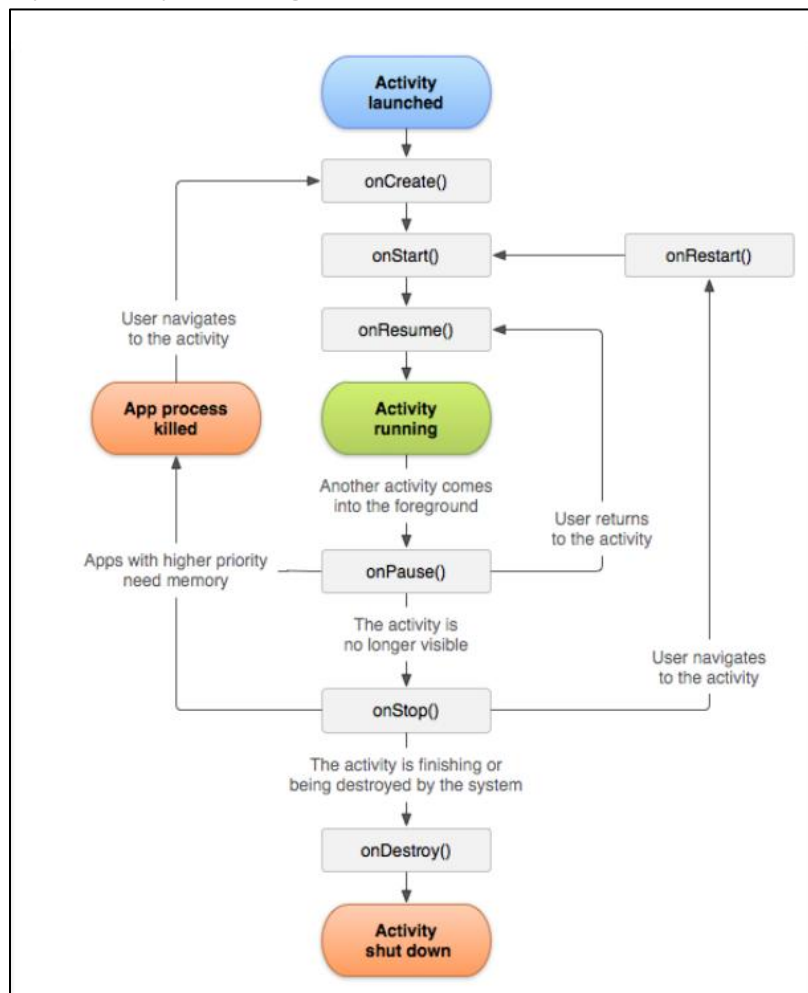
6. onRestart()

Called when an activity is coming back to the foreground after being stopped. It allows the activity to re-initialize resources.

7. onDestroy()

Called before the activity is destroyed, either due to user finishing the activity or system reclaiming resources. Cleanup should be done here.

Activity Life Cycle Diagram



3. Define Fragment. Discuss the lifecycle of a fragment

ANS:- Introduction

In Android, a Fragment is a modular section of an activity that has its own lifecycle, layout, and behavior. Fragments are designed to create flexible and reusable UI components that can be combined in different activities, enabling developers to design dynamic and adaptable user

interfaces. They are particularly useful for applications targeting tablets or large-screen devices, where multiple fragments can be displayed side by side within the same activity.

What is a Fragment

A Fragment represents a portion of the user interface in an activity. Unlike activities, fragments cannot exist independently; they must be hosted within an activity. Fragments have their own lifecycle methods, can handle user interactions, and can communicate with other fragments or the parent activity. This modular approach allows developers to reuse fragments across multiple activities, enhancing maintainability and reducing code duplication.

Example:

```
public class ExampleFragment extends Fragment {  
    @Override  
    public View onCreateView(LayoutInflater inflater, ViewGroup container,  
        Bundle savedInstanceState) {  
        return inflater.inflate(R.layout.fragment_example, container, false);  
    }  
}
```

Here, ExampleFragment inflates its own layout, fragment_example.xml, within the parent activity.

Fragment Lifecycle

The Fragment Lifecycle defines the stages a fragment goes through from creation to destruction. Understanding these stages is essential for resource management, data persistence, and user interface updates. The main lifecycle methods include:

1. onAttach()

Called when the fragment is associated with its parent activity. This is the first step in the fragment lifecycle.

2. onCreate()

Invoked to initialize the fragment, create variables, and set up configurations. The fragment is created but UI elements are not yet initialized.

3. onCreateView()

Called to inflate the fragment's layout. Here, the user interface elements are created and prepared for display.

4. onActivityCreated()

Invoked after the parent activity's onCreate() is complete. It allows the fragment to access and interact with the activity's UI components.

5. onStart()

Called when the fragment becomes visible to the user. The fragment is now in the foreground but not necessarily interactive.

6. onResume()

Called when the fragment is active and interacting with the user. This is the state where UI updates and animations can be performed.

7. onPause()

Invoked when the fragment is about to go into the background. Save unsaved data or pause ongoing tasks here.

8. onStop()

Called when the fragment is no longer visible. Stop tasks that do not need to run while the fragment is hidden.

9. onDestroyView()

Called when the fragment's view hierarchy is being removed. Release resources tied to the UI here.

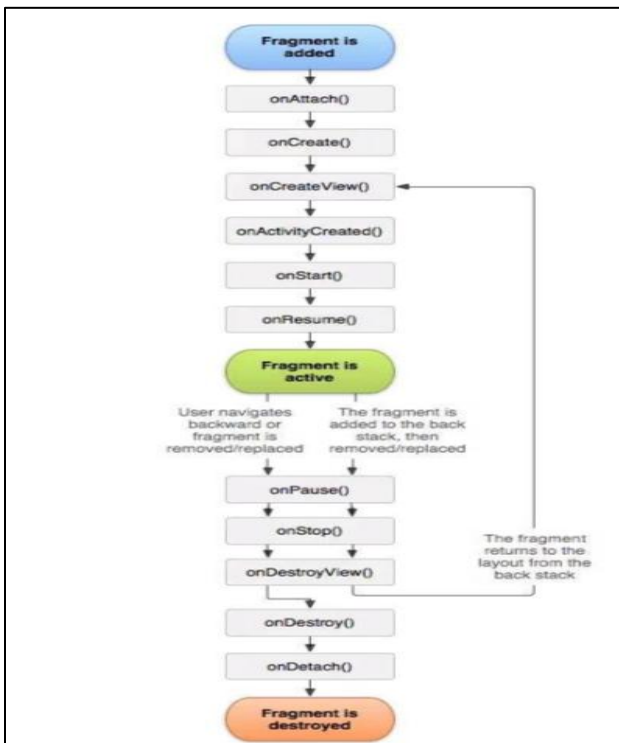
10. onDestroy()

Invoked to perform final cleanup before the fragment is destroyed. Release non-UI resources here.

11. onDetach()

Called when the fragment is disassociated from its activity. This is the final stage of the fragment lifecycle.

Fragment Lifecycle Diagram



4. Which Dialog Boxes are supported by Android? Explain Alert Dialog with its methods and attributes.

ANS:- Introduction

In Android, a dialog box is a small window that prompts the user to make a decision or enter additional information. Dialogs are temporary windows that appear in front of the current activity, allowing users to interact without leaving the current screen. Android supports several types of dialog boxes, each designed for specific use cases, such as displaying information, asking confirmation, or obtaining input.

Types of Dialog Boxes Supported by Android

Android supports the following dialog types:

1. **Alert Dialog** – Used to display messages and get user confirmation.
2. **Date Picker Dialog** – Allows the user to select a date from a calendar interface.
3. **Time Picker Dialog** – Enables the user to choose a time.
4. **Progress Dialog** – Displays a progress bar for tasks running in the background.
5. **Custom Dialog** – Allows developers to create a dialog with a custom layout for specialized UI requirements.

Among these, the Alert Dialog is the most commonly used because it is flexible, interactive, and easy to implement.

Alert Dialog in Android

An Alert Dialog is a dialog box that presents information and requests user action. It can display a title, message, buttons, and optionally icons or custom views. Alert Dialogs are widely used for confirming actions, showing warnings, or providing choices.

Creating an AlertDialog

To create an alert dialog, developers use the `AlertDialog.Builder` class, which provides methods for configuring the dialog content, buttons, and behavior.

Example:

```
AlertDialog.Builder builder = new AlertDialog.Builder(this);
builder.setTitle("Exit App");
builder.setMessage("Are you sure you want to exit?");
builder.setPositiveButton("Yes", new DialogInterface.OnClickListener() {
    public void onClick(DialogInterface dialog, int id) {
        finish(); // Close activity
    }
});
builder.setNegativeButton("No", null);
AlertDialog dialog = builder.create();
dialog.show();
```

Important Methods of Alert Dialog

1. **setTitle(String title)** – Sets the title of the dialog.
2. **setMessage(String message)** – Sets the main content message.
3. **setIcon(int resId)** – Adds an icon to the dialog for emphasis.
4. **setPositiveButton(String text, OnClickListener listener)** – Adds a positive action button.

5. **setNegativeButton(String text, OnClickListener listener)** – Adds a negative action button.
6. **setNeutralButton(String text, OnClickListener listener)** – Adds a neutral action button.
7. **create()** – Builds the AlertDialog object.
8. **show()** – Displays the dialog to the user.

Important Attributes of Alert Dialog

- **Title** – Displays the main heading of the dialog.
- **Message** – The content text providing information to the user.
- **Icon** – Optional graphical representation for emphasis.
- **Cancelable** – Determines whether the user can dismiss the dialog by tapping outside or pressing the back button.
- **Buttons** – Positive, negative, and neutral buttons allow the user to respond to the dialog.
- **Custom View** – Optional layout can be set to replace or enhance the default dialog UI.

Example with Attributes:

```
builder.setCancelable(false); // User cannot dismiss by tapping outside
builder.setIcon(R.drawable.warning_icon); // Set dialog icon
```

5. Explain the purpose and usage of Button & Toggle Button widgets in Android.

ANS:- Introduction

In Android, widgets are UI elements that allow users to interact with the application. Among them, Button and Toggle Button are commonly used for triggering actions and providing binary choices. Understanding their purpose, usage, and attributes is essential for designing interactive and responsive user interfaces.

Button Widget

A Button is a clickable UI element that performs an action when the user taps on it. Buttons are widely used for tasks such as submitting forms, navigating between screens, or triggering specific functions in an application. They can be customized with text, images, colors, and background styles to fit the app's design.

Example:

```
<Button
```

```
    android:id="@+id/submitButton"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Submit"/>
```

Usage in Java:

```
Button submitButton = findViewById(R.id.submitButton);
submitButton.setOnClickListener(new View.OnClickListener() {
    @Override
    public void onClick(View v) {
```

```

        Toast.makeText(getApplicationContext(), "Button Clicked",
Toast.LENGTH_SHORT).show();
    }
});

```

Purpose:

- Trigger specific actions when clicked.
- Enable navigation between activities.
- Submit data or forms.
- Provide visual feedback using state changes or animations.

Toggle Button Widget

A Toggle Button is a two-state button that can switch between ON and OFF states. It is used to represent a binary choice or control a setting, such as enabling/disabling Wi-Fi or activating silent mode. Toggle Buttons provide immediate visual feedback indicating the current state.

Example:

```

<ToggleButton
    android:id="@+id/toggleButton"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:textOn="ON"
    android:textOff="OFF"/>

```

Usage in Java:

```

ToggleButton toggleButton = findViewById(R.id.toggleButton);
toggleButton.setOnCheckedChangeListener(new
CompoundButton.OnCheckedChangeListener() {
    @Override
    public void onCheckedChanged(CompoundButton buttonView, boolean isChecked) {
        if(isChecked) {
            Toast.makeText(getApplicationContext(), "Toggle ON",
Toast.LENGTH_SHORT).show();
        } else {
            Toast.makeText(getApplicationContext(), "Toggle OFF",
Toast.LENGTH_SHORT).show();
        }
    }
});

```

Purpose:

- Represent binary options clearly to users.
- Allow quick toggling of settings or features.

- Provide visual state feedback using text, color, or images.
- Enable interactive and intuitive controls in applications.

Comparison of Button and Toggle Button

Feature	Button	Toggle Button
States	Single state	Two states (ON/OFF)
Purpose	Trigger actions	Switch settings or binary options
Feedback	Visual click feedback	Visual state feedback (ON/OFF)
Use Case	Submitting a form, navigation	Enabling/disabling features like Wi-Fi or sound

6. What is the use of the Progress Bar widget?

ANS:- Introduction

In Android, a Progress Bar is a visual indicator that shows the progress of a task. It is commonly used to inform users about ongoing operations, such as file downloads, data uploads, or background processing, where the exact completion time may not be immediately known. Progress Bars improve user experience by providing feedback about the status of long-running tasks, preventing users from assuming the app has frozen.

Purpose of Progress Bar

The primary purposes of a Progress Bar are:

- **Indicate Task Progress:** Shows how much of a task is completed.
- **Enhance User Experience:** Prevents user frustration by providing visual feedback.
- **Track Background Processes:** Useful for tasks like loading files, network operations, or database queries.
- **Provide Time Estimates:** Communicates that a process is ongoing and may take some time.

Types of Progress Bar

1. **Determinate Progress Bar** – Displays specific progress (percentage completed) of a task.
2. **Indeterminate Progress Bar** – Shows continuous animation when the exact progress is unknown, indicating that the task is in progress.

Creating a Progress Bar

Example: Determinate Progress Bar

```
<ProgressBar
```

```
    android:id="@+id/progressBar"
```

```
    style="@android:style/Widget.ProgressBar.Horizontal"
```

```
    android:layout_width="match_parent"
```



```
android:layout_height="wrap_content"
android:max="100"
android:progress="0"/>
```

Usage in Java:

```
ProgressBar progressBar = findViewById(R.id.progressBar);
progressBar.setProgress(50); // Set progress to 50%
```

Example: Indeterminate Progress Bar

```
<ProgressBar
    android:id="@+id/progressBarIndeterminate"
    style="@android:style/Widget.ProgressBar.Small"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:indeterminate="true"/>
```

Attributes of Progress Bar

- **max** – Maximum value of the progress bar.
- **progress** – Current progress value.
- **indeterminate** – Boolean indicating whether the progress is determinate or indeterminate.
- **visibility** – Determines whether the progress bar is visible or hidden.
- **style** – Defines the appearance of the progress bar (horizontal, small, large, etc.).

7. Explain Rating Bar in Android.

ANS:- **Rating Bar in Android**

Introduction

In Android, a Rating Bar is a widget that allows users to provide feedback or rating for an item, service, or application. It is commonly represented as a row of stars or other symbols, where users can select the number of stars to indicate their rating. Rating Bars are widely used in apps, online shopping, media platforms, and feedback forms to gather user opinions and preferences.

Purpose of Rating Bar

The primary purposes of a Rating Bar are:

- **Collect User Feedback:** Allows users to rate products, services, or content.
- **Visual Representation:** Provides a clear, intuitive visual interface for rating.
- **Interactive Control:** Users can set or modify ratings easily.
- **Data Collection:** Helps developers analyze user satisfaction and improve app features.

Creating a Rating Bar

A Rating Bar can be added in XML layout and customized with attributes such as the number of stars, step size, and default rating.

Example:

```
<RatingBar
    android:id="@+id/ratingBar"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:numStars="5"
    android:stepSize="1"
    android:rating="3"/>
```

Usage in Java:

```
RatingBar ratingBar = findViewById(R.id.ratingBar);
float rating = ratingBar.getRating(); // Get current rating
ratingBar.setOnRatingBarChangeListener(new RatingBar.OnRatingBarChangeListener() {
    @Override
    public void onRatingChanged(RatingBar ratingBar, float rating, boolean fromUser) {
        Toast.makeText(getApplicationContext(), "Rating: " + rating,
        Toast.LENGTH_SHORT).show();
    }
});
```

Attributes of Rating Bar

- **numStars** – Specifies the total number of stars displayed.
- **rating** – Sets the current rating.
- **stepSize** – Determines the increment value when changing the rating (e.g., 0.5 or 1).
- **isIndicator** – Boolean that determines if the rating bar is read-only.
- **visibility** – Controls whether the rating bar is visible or hidden.

8. Explain the commonly used Android UI widgets for user input and display, such as text fields, selection controls, and interactive pickers. Give examples and their typical use cases.

ANS:- Android UI widgets are the building blocks of interactive applications. EditText and AutoCompleteTextView capture textual input efficiently. CheckBox, RadioButton, and Spinner allow users to make selections easily. DatePicker and TimePicker provide convenient ways to choose dates and times, while ImageView enhances the visual interface by displaying images. Using these widgets effectively improves user experience and makes Android apps functional and user-friendly.

Android UI Widgets for User Input and Display

1. Text Input Widgets

Text input widgets allow users to enter and edit textual data in an Android application. They are essential for forms, search fields, and other interactive elements.

- **EditText:**

EditText provides a field where users can enter single-line or multi-line text. It supports different input types such as plain text, numeric, email, or password.

Example Code:

- `<EditText`
`android:layout_width="match_parent"`
`android:layout_height="wrap_content"`
`android:hint="Enter username"/>`

AutoCompleteTextView:

AutoCompleteTextView extends EditText by providing suggestions based on user input. It improves user experience and reduces typing effort.

Code:

```
AutoCompleteTextView acText = findViewById(R.id.autoText);
acText.setAdapter(new ArrayAdapter<>(this, android.R.layout.simple_list_item_1,
items));
```

2. Selection Controls

Selection controls allow users to select one or multiple options. They are widely used in surveys, forms, and settings.

- **CheckBox:**

CheckBox allows multiple selections from a list of options. Each CheckBox works independently.

Code:

```
<CheckBox android:text="Music"/>
```

- **RadioButton:**

RadioButton allows only a single selection within a RadioGroup, ensuring mutual exclusivity.

Code:

```
<RadioButton android:text="Male"/>
```

- **Spinner:**

Spinner displays a dropdown list for selecting one option. It saves space compared to multiple RadioButtons.

Code:

```
Spinner spinner = findViewById(R.id.spinner);
spinner.setAdapter(new ArrayAdapter<>(this, android.R.layout.simple_spinner_item,
items));
```

3. Pickers for Date and Time

Pickers simplify the selection of dates and times, providing a consistent user interface.

- **DatePicker:**

DatePicker allows the user to select a date from a calendar view. It is commonly used for birthdates, deadlines, or event scheduling.

Code:

```
<DatePicker android:id="@+id/datePicker"/>
```

- **TimePicker:**

TimePicker provides an interface to select hours and minutes, supporting both 12-hour and 24-hour formats.

Code:

```
<TimePicker android:id="@+id/timePicker"/>
```

4. Display Widget

- **ImageView:**

ImageView is used to display images in an application. It supports multiple formats like PNG, JPEG, and GIF, and can be scaled or cropped as needed.

Code:

```
<ImageView android:src="@drawable/profile"/>
```

9. What is Web View? List out any methods of Web View class and explain how to Load any simple HTML file on WebView.

ANS:- Definition:

WebView is an Android UI widget that allows applications to display web pages or HTML content directly within the app interface. It acts like a mini-browser embedded in the application, so users do not need to open an external browser. WebView is useful for showing online content, local HTML pages, or any simple web-based interface in Android apps. Typical applications include displaying terms and conditions, help pages, tutorials, or advertisements.

Features of WebView:

1. Can display both local HTML files and web pages from the internet.
2. Supports JavaScript execution when enabled.
3. Allows navigation within the WebView without opening external browsers.
4. Can handle user interaction like clicks, form submission, and scrolling.
5. Supports loading of custom HTML, CSS, and JavaScript content.

Common Methods of WebView Class:

1. **loadUrl(String url):**
Loads a web page from a specified URL. It can be an online website or a local HTML file in the assets folder.
2. **loadData(String data, String mimeType, String encoding):**
Loads HTML content directly from a string instead of a file or URL. Useful for dynamically generating web content.
3. **getSettings():**
Returns a WebSettings object that allows configuration of WebView properties such as enabling JavaScript, zoom controls, or caching.
4. **setWebViewClient(WebViewClient client):**
Ensures that page navigation (like clicking links) happens inside the WebView instead of opening an external browser.
5. **canGoBack():**
Checks whether the WebView can navigate back in history. Useful for handling the back button.
6. **goBack():**
Navigates back to the previous page in WebView history.

Loading a Simple HTML File in WebView:

Step 1 – Layout XML:

```
<WebView  
    android:id="@+id/myWebView"  
    android:layout_width="match_parent"  
    android:layout_height="match_parent"/>
```

This XML code defines a WebView widget that fills the screen. The id is used to reference it in the Java/Kotlin code.

Step 2 – Java Code:

```
WebView myWebView = findViewById(R.id.myWebView);  
myWebView.getSettings().setJavaScriptEnabled(true); // Enable JavaScript if needed  
myWebView.setWebViewClient(new WebViewClient()); // Keep navigation within WebView  
myWebView.loadUrl("file:///android_asset/sample.html"); // Load local HTML file
```

Explanation of Code:

- `getSettings().setJavaScriptEnabled(true)` allows execution of JavaScript in the HTML page, which is necessary if the page contains scripts.
- `setWebViewClient(new WebViewClient())` ensures that all links clicked within the WebView open inside the app, preventing the system browser from opening.
- `loadUrl("file:///android_asset/sample.html")` loads a local HTML file named `sample.html` stored in the assets folder. The same method can be used to load online pages by replacing the path with a URL, e.g., `"https://www.example.com"`.

Step 3 – Additional Settings (Optional):

- **Zoom Controls:** Enable pinch-to-zoom for better user experience.
- **Caching:** Improve performance by caching frequently accessed pages.
- **Client Handling:** Customize `WebViewClient` to handle page errors or redirects.

Advantages of WebView:

1. Allows integration of web content without leaving the app.
2. Supports dynamic and interactive content, including HTML, CSS, and JavaScript.
3. Useful for hybrid applications that combine native and web-based UI.
4. Enables easy display of local HTML resources, which reduces internet dependency.

UNIT 3

1. Define Menu? Explain different types of menus

ANS:- Introduction

In Android, a Menu is a vital component of the user interface that provides a list of options or actions for the user to interact with. Menus help in organizing commands, settings, navigation options, or user actions in a structured manner. Instead of placing too many buttons on the screen, Android uses menus to save space and improve usability. Menus maintain consistency across activities and are widely used in applications such as settings, file options, sharing, profile actions, and shortcuts.

Menus are typically defined in XML resource files and then inflated in activities or fragments using the `onCreateOptionsMenu()` method. Android provides multiple types of menus, each designed for different user interaction patterns.

What is a Menu?

A Menu in Android is a set of user interface options or commands that appear in the form of a list or popup. It enables users to execute specific actions like opening a new screen, saving data, searching content, changing settings, or exiting the application. Menus improve the overall navigation experience and make the application more organized and user-friendly.

Types of Menus in Android

Android supports several built-in types of menus, each serving different purposes based on the interaction model and screen space.

1.Options Menu

The Options Menu is the primary menu of an activity and appears in the app bar (toolbar). It is used for global actions such as search, settings, share, or help. It usually appears as a 3-dot icon (overflow menu) on the top-right of the screen when there are multiple menu items.

- It is created using `onOptionsItemSelected()` method.
- It can have icons or text items.
- It is visible for the entire activity.

Common use cases: Settings, Logout, About, Search, Help

Context Menu

A Context Menu appears when the user performs a long press (long-click) on a view. It provides context-specific actions related to the selected item. Unlike the Options Menu, the Context Menu is used only for a particular item, not for the entire activity.

- It is created using `onCreateContextMenu()` method.
- It provides actions like copy, paste, delete, rename.
- Appears as a floating menu near the selected view.

Common use cases: Editing text, managing list items, file operations

Popup Menu

A Popup Menu displays a small list of actions anchored to a specific view. It appears next to the view when the user taps a button. Popup Menus are lightweight and temporary, and they disappear automatically after the user selects an option or taps elsewhere.

- It is created using `PopupMenu` class in Java/Kotlin.
- It is not tied to the Activity's lifecycle.
- It is visually similar to a dropdown menu.

Common use cases: Overflow actions in cards, sorting options, quick actions on list items

Contextual Action Mode (ActionMode Menu)

Contextual Action Mode is a temporary action bar displayed at the top of the screen when the user selects an item. It is commonly used in multi-select lists or file managers.

- Activated on long press or by selecting multiple items.
- Displays an action bar with contextual actions like delete, share, select all.
- Ends when the user clears the selection or the action is completed.

Common use cases: Email apps, gallery apps, file selection, batch operations

Sub Menu

A Sub Menu is a menu inside another menu item. It allows hierarchical organization of actions and is used when there are multiple related options.

- It is created using `SubMenu` class.

- Appears as a nested dropdown from a parent menu item.
- Useful for grouped settings or categories.

Common use cases: Display settings → Brightness, Font size, Theme, etc.

2. Explain SQLite Database class with its methods.

ANS:- Introduction

In Android, SQLite is a lightweight, relational database engine that is embedded within the operating system. It allows developers to store, manage, and retrieve structured data locally on the device without needing an external server. Android provides the SQLiteDatabase class to perform database operations such as creating tables, inserting data, updating records, deleting data, and executing SQL queries. The database is stored in the internal storage of the application, making it secure and private to that specific app.

To make working with SQLite easier, Android commonly uses the SQLiteOpenHelper class to create and upgrade databases, while the SQLiteDatabase class performs the actual database operations. This class is efficient, fast, and widely used in applications like note-taking apps, login systems, offline storage, and caching.

SQLiteDatabase Class

The SQLiteDatabase class provides methods to open, query, insert, update, delete, and manage transactions in an SQLite database. It supports both readable and writable database modes.

When getWritableDatabase() is called, an instance of SQLiteDatabase is returned to allow full access to the database.

This class supports two types of SQL commands:

- **DDL (Data Definition Language)** – CREATE, ALTER, DROP
- **DML (Data Manipulation Language)** – INSERT, UPDATE, DELETE, SELECT

Common Methods of SQLiteDatabase Class

Below are the most frequently used methods of the SQLiteDatabase class in Android:

1. execSQL(String sql)

This method is used to execute a single SQL statement that does not return any data. It is commonly used for creating or altering tables and inserting, updating, or deleting records without expecting a result.

Example:

```
db.execSQL("CREATE TABLE users(id INTEGER PRIMARY KEY, name TEXT)");
```

2. insert(String table, String nullColumnHack, ContentValues values)

This method is used to insert a new row into a table. It accepts:

- Table name
- A null column hack (to avoid empty row errors)
- A ContentValues object containing column-value pairs

It returns the row ID of the newly inserted row or -1 if failed.

3. update(String table, ContentValues values, String whereClause, String[] whereArgs)

This method is used to modify existing records in a table.

- values: Column-value pairs to update
- whereClause: Filter condition (e.g., "id=?")
- whereArgs: Values for the condition

It returns the number of rows affected.

4. delete(String table, String whereClause, String[] whereArgs)

This method is used to delete rows from a table based on a condition.

- If whereClause is null, all rows will be deleted.
- It returns the number of rows deleted.

5. query(...)

This method is used to retrieve data from the database using a SELECT statement. It supports parameters like table name, columns, selection, grouping, and sorting. It returns a Cursor object which allows iteration over results.

Typical parameters include:

- Table name
- Columns
- Selection and selectionArgs
- GroupBy, Having, OrderBy

Example:

```
Cursor cursor = db.query("users", null, null, null, null, null, null);
```

6. rawQuery(String sql, String[] selectionArgs)

This method allows execution of custom SELECT queries in SQL format.

Example:

```
Cursor cursor = db.rawQuery("SELECT * FROM users WHERE id=?", new String[]{"1"});
```

It also returns a Cursor for result handling.

7. beginTransaction(), setTransactionSuccessful(), endTransaction()

SQLite supports transactions to ensure data integrity.

- beginTransaction() – Starts a transaction
- setTransactionSuccessful() – Marks the transaction as successful
- endTransaction() – Ends the transaction (commits if successful, otherwise rolls back)

These methods ensure atomic operations and prevent partial updates.

8. close()

This method is used to close the SQLite database connection. It is important to call this method to free system resources and avoid memory leaks.

3. Write an Android Application to send Email.

ANS:- A simple Android app code that opens the email client and sends an email using Intent.ACTION_SEND.

1.activity_main.xml (UI Layout)


```

<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:padding="16dp">
    <EditText
        android:id="@+id/etTo"
        android:hint="Enter Recipient Email"
        android:layout_width="match_parent"
        android:layout_height="wrap_content" />
    <EditText
        android:id="@+id/etSubject"
        android:hint="Enter Subject"
        android:layout_width="match_parent"
        android:layout_height="wrap_content" />
    <EditText
        android:id="@+id/etMessage"
        android:hint="Enter Message"
        android:gravity="top"
        android:lines="5"
        android:layout_width="match_parent"
        android:layout_height="wrap_content" />
    <Button
        android:id="@+id/btnSend"
        android:text="Send Email"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content" />
</LinearLayout>

```

2. MainActivity.java

```

package com.example.sendemail;
import androidx.appcompat.app.AppCompatActivity;
import android.content.Intent;
import android.net.Uri;
import android.os.Bundle;
import android.view.View;
import android.widget.Button;
import android.widget.EditText;
import android.widget.Toast;

```

```

public class MainActivity extends AppCompatActivity {

    EditText etTo, etSubject, etMessage;
    Button btnSend;
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        etTo = findViewById(R.id.etTo);
        etSubject = findViewById(R.id.etSubject);
        etMessage = findViewById(R.id.etMessage);
        btnSend = findViewById(R.id.btnSend);
        btnSend.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View v) {
                String to = etTo.getText().toString();
                String subject = etSubject.getText().toString();
                String message = etMessage.getText().toString();
                if(to.isEmpty()){
                    Toast.makeText(MainActivity.this, "Please enter recipient",
Toast.LENGTH_SHORT).show();
                } else {
                    Intent emailIntent = new Intent(Intent.ACTION_SENDTO);
                    emailIntent.setData(Uri.parse("mailto:" + to));
                    emailIntent.putExtra(Intent.EXTRA_SUBJECT, subject);
                    emailIntent.putExtra(Intent.EXTRA_TEXT, message);
                    try {
                        startActivity(Intent.createChooser(emailIntent, "Send Email Using..."));
                    } catch (android.content.ActivityNotFoundException ex) {
                        Toast.makeText(MainActivity.this, "No email clients installed.",
Toast.LENGTH_SHORT).show();
                    }
                }
            }
        });
    }
}

```

4. Explain the features and functions of Telephony Manager API.

ANS:- Introduction

The Telephony Manager API in Android is a system service that provides access to information about the telephony services on the device. It allows applications to interact with and obtain details from the mobile network, SIM card, and device-related telephony features. The API is designed to provide read-only information such as network type, device identifiers, SIM details, and call state. It can also be used to monitor telephony events through listener classes. The TelephonyManager class is part of the Android framework and can be obtained using the getSystemService() method with the parameter TELEPHONY_SERVICE. This enables applications to access essential device and network information that is critical for communication-related applications.

The TelephonyManager API serves multiple purposes. It helps applications to determine network availability, check SIM card readiness, identify the operator and country, and monitor call states and signal strength. This API is essential for applications like mobile network monitors, call management apps, and SIM-based authentication systems.

Features of Telephony Manager API

Access Device & Subscriber Information

- Methods like getDeviceId() return the IMEI or MEID number of the device.
- getSubscriberId() provides the IMSI associated with the SIM card.
- getSimSerialNumber() and getLine1Number() retrieve the SIM's serial number and the mobile number, respectively.
- These methods are crucial for device identification or user authentication.

Network Information

- getNetworkOperatorName() returns the name of the mobile network operator.
- getNetworkType() indicates the current network technology such as 2G, 3G, 4G, or 5G.
- getNetworkCountryIso() provides the ISO code of the country in which the network operates.
- Useful for applications that adapt behavior according to network type or geographic location.

SIM Card Management

- getSimOperatorName(), getSimState(), and getSimCountryIso() provide SIM operator name, status, and country.
- Important for apps offering dual-SIM support or subscription-based services.

Monitoring Call and Data State

- getCallState() detects whether the device is idle, ringing, or in a call.
- getDataState() indicates if mobile data is connected, disconnected, or connecting.
- isNetworkRoaming() checks if the device is roaming.

- PhoneStateListener allows real-time monitoring of telephony events, including call state changes, data activity, signal strength, and cell location changes.

Methods of TelephonyManager

Method	Purpose
getDeviceId()	Returns IMEI / MEID
getSubscriberId()	Returns IMSI
getLine1Number()	Returns mobile number
getNetworkOperatorName()	Returns operator name
getNetworkType()	Returns current network type
getSimState()	Returns SIM status
getCallState()	Returns call state
getDataState()	Returns data connection state
isNetworkRoaming()	Checks if device is roaming
listen()	Registers PhoneStateListener for updates

Applications

- Display network and SIM information in apps.
- Track call and data usage.
- Manage dual-SIM devices.
- Used in mobile network monitoring apps, call-blocking apps, telecom authentication, and billing apps.
- Allows applications to adapt dynamically to changes in network and telephony conditions.

5. List two techniques to send SMS on mobile devices. Explain any one in detail.

ANS:- Techniques to Send SMS on Mobile Devices

In Android, sending SMS (Short Message Service) is a common feature for communication applications. There are two main techniques to send SMS on mobile devices:

1. Using SmsManager Class
2. Using Intent with ACTION_SENDTO

1. Using SmsManager Class

The SmsManager class allows applications to send SMS messages directly from the device, without opening the default messaging app. This method provides full control over sending messages, including sending multipart messages, scheduled messages, and monitoring delivery status.

Steps to Send SMS using SmsManager:

1. Obtain SmsManager instance

```
SmsManager smsManager = SmsManager.getDefault();
```

2. Send a text message

```
smsManager.sendTextMessage(  
    "recipient_number", // Phone number of recipient  
    null,               // Service center address (null for default)  
    "Hello! This is a test SMS", // Message body  
    null,               // PendingIntent for sent SMS (optional)  
    null                // PendingIntent for delivery (optional)  
);
```

3. Permissions Required

To send SMS directly, the following permission must be added to AndroidManifest.xml:

```
<uses-permission android:name="android.permission.SEND_SMS" />
```

4. Runtime Permission

From Android 6.0 (API 23) onward, runtime permission is also required to ensure user consent for sending messages.

Features of SmsManager Method:

- Sends SMS without opening the messaging app
- Can handle long or multipart messages
- Can provide delivery status using PendingIntent
- Works for automated messages or app-controlled notifications

Example with Sent and Delivery Confirmation:

```
PendingIntent sentPI = PendingIntent.getBroadcast(this, 0, new Intent("SMS_SENT"), 0);  
PendingIntent deliveredPI = PendingIntent.getBroadcast(this, 0, new  
Intent("SMS_DELIVERED"), 0);
```

```
smsManager.sendTextMessage("1234567890", null, "Hello SMS", sentPI, deliveredPI);
```

2. Using Intent with ACTION_SENDTO

This technique opens the default messaging app with the recipient number and message pre-filled. The user must manually press send.

```
Intent smsIntent = new Intent(Intent.ACTION_SENDTO);  
smsIntent.setData(Uri.parse("smsto:1234567890"));  
smsIntent.putExtra("sms_body", "Hello SMS");  
startActivity(smsIntent);
```

Advantages:

- No need for SEND_SMS permission
- Safer and user-friendly
- Uses default SMS app for sending

Disadvantages:

- User interaction is required to send
- Cannot automate sending

UNIT 4

1. What is an Android sensor? What are the types of sensors?

ANS:- Android Sensor

An Android sensor is a hardware or software component in an Android device that measures and provides data about the device's environment or physical movement. Sensors allow applications to detect motion, orientation, environmental conditions, and user interactions. The Android platform provides a sensor framework that gives access to these sensors, enables data collection, and allows applications to monitor changes in real time.

Sensors are commonly used in applications such as gaming, fitness tracking, navigation, screen rotation, augmented reality, and environmental monitoring. The sensor framework includes classes and interfaces like `SensorManager`, `Sensor`, and `SensorEventListener`, which help developers register sensors, receive updates, and manage sensor accuracy.

The primary advantages of sensors in Android are improved user experience, automatic behavior based on movement or environment, and support for smart, context-aware applications. Sensors can be hardware-based, which directly access the physical sensor chip, or software-based, which calculate values using algorithms and other hardware sensors (for example, orientation sensor derived from accelerometer and magnetometer).

Types of Android Sensors

Android sensors are mainly classified into three major categories:

1. Motion Sensors

Motion sensors measure acceleration, rotation, and movement of the device. These sensors are essential in gaming, fitness apps, gesture detection, shake detection, and step counting.

Common motion sensors:

- **Accelerometer** – Measures acceleration force applied to the device on x, y, z axes. Used for screen rotation and motion detection.
- **Gyroscope** – Measures rate of rotation around the axes. Used for precision motion, gaming, and VR applications.
- **Gravity Sensor** – Measures the force of gravity on the device.
- **Linear Acceleration Sensor** – Measures acceleration minus gravity.
- **Rotation Vector Sensor** – Provides the device orientation using a combination of sensors.

These sensors help detect tilt, shake, spin, and directional movement.

2. Position Sensors

Position sensors determine the physical position of the device relative to the Earth's magnetic field or nearby objects. They are used in navigation, compass apps, proximity detection, and location-based services.

Common position sensors:

- **Magnetic Field Sensor** – Measures the geomagnetic field in micro-Tesla. Used to build digital compass.
- **Proximity Sensor** – Detects how close an object is to the screen (e.g., detects face during a call to turn off the display).

- **Orientation Sensor (Deprecated)** – Provided device orientation based on accelerometer and magnetometer. Replaced by rotation vector sensor.

Position sensors help in determining direction, device alignment, and proximity-based actions.

3. Environmental Sensors

Environmental sensors measure physical environmental factors such as temperature, pressure, humidity, and light. These sensors are used in weather apps, science tools, environmental monitoring, and battery optimization.

Common environmental sensors:

- **Ambient Light Sensor** – Measures light intensity. Used for automatic brightness control.
- **Barometer (Pressure Sensor)** – Measures atmospheric pressure. Helps improve GPS altitude accuracy and weather apps.
- **Ambient Temperature Sensor** – Measures temperature of the environment.
- **Relative Humidity Sensor** – Measures the amount of moisture in the air.

These sensors provide real-world environmental data to applications.

2. Write a short note on Bluetooth in android

ANS:- **Bluetooth in Android**

Bluetooth in Android is a short-range wireless communication technology used to exchange data between devices such as smartphones, headsets, speakers, wearables, and IoT devices. It operates typically within a range of 10 meters and consumes low power, making it suitable for continuous background connectivity.

Android provides a complete Bluetooth API that allows developers to perform various operations like enabling/disabling Bluetooth, making the device discoverable, scanning for nearby devices, pairing, and establishing connections for data transfer.

Features of Bluetooth in Android

1. **Wireless Data Transfer** – Allows sending/receiving text, files, audio, etc.
2. **Low Power Consumption** – Efficient for continuous connectivity.
3. **Supports Multiple Profiles** – A2DP (audio), HID (input), PAN, etc.
4. **Secure Pairing and Encryption** – Protects data during transmission.
5. **Device-to-Device Communication** – No internet required.
6. **Background Operation** – Works even when app is minimized.

Functions / Capabilities of Bluetooth API

1. **Check Bluetooth Availability**
 - `BluetoothAdapter.getDefaultAdapter()`
2. **Enable or Disable Bluetooth**
 - `bluetoothAdapter.enable()` or via Intent
3. **Make Device Discoverable**
 - Using `ACTION_REQUEST_DISCOVERABLE` intent
4. **Scan for Nearby Devices**
 - `startDiscovery()` method
5. **Pair / Bond Devices**

- createBond() method

6. Establish Connection

- Using BluetoothSocket and RFCOMM channels

7. Send / Receive Data

- InputStream and OutputStream over socket

8. Manage Connected Devices

- getBondedDevices()

Common Use Cases

- Wireless file sharing
- Connecting audio devices (headphones, speakers)
- Bluetooth calling in cars
- Fitness trackers and smart watches
- Multiplayer games (peer-to-peer)
- IoT device communication

Advantages

- No internet required
- Secure and encrypted
- Energy efficient
- Universally supported

Limitations

- Short range (10–100m)
- Slower speed compared to Wi-Fi
- Limited number of simultaneous connections

3. What is animation? List out different types of animation and explain anyone.

ANS:- Animation in android

animation in android refers to the process of applying gradual visual changes to ui elements over a specific duration to make the interface more interactive and appealing. It enhances user experience by providing motion, feedback, and smooth transitions between user actions.

Animations help in drawing attention, improving navigation, and visually indicating changes in the state of views.

Purpose of using animation

- To make ui attractive and user-friendly
- To provide smooth transitions between screens or views
- To show changes in state or visibility
- To guide user attention to important elements
- To simulate real-world motion for better interaction

Types of animation in android

android supports multiple animation techniques. The commonly used ones are:

1. Fade animation (fade in / fade out)
2. Zoom animation (zoom in / zoom out)
3. Rotate animation
4. Translate (move) animation
5. Slide animation
6. Blink animation
7. Bounce animation
8. Frame-by-frame animation
9. Property animation (objectanimator / valueanimator)
10. Animation set (combining multiple animations)

Fade animation

fade animation is used to gradually increase or decrease the visibility of a view by changing its alpha (opacity) value.

- **Fade in:** makes a view appear slowly by changing alpha from 0.0 to 1.0
- **Fade out:** makes a view disappear slowly by changing alpha from 1.0 to 0.0

This type of animation is often used when displaying text messages, images, or showing/hiding elements smoothly without abrupt changes.

Xml example for fade in (fade_in.xml):

```
<alpha xmlns:android="http://schemas.android.com/apk/res/android"
    android:fromalpha="0.0"
    android:toalpha="1.0"
    android:duration="1000"/>
```

Xml example for fade out (fade_out.xml):

```
<alpha xmlns:android="http://schemas.android.com/apk/res/android"
    android:fromalpha="1.0"
    android:toalpha="0.0"
    android:duration="1000"/>
```

Applying animation in activity:

```
Animation fade = animationutils.loadanimation(this, r.anim.fade_in);
View.startanimation(fade);
```

Advantages of fade animation

- Simple to implement
- Provides smooth visual transitions
- Works on any ui component
- Does not require layout modifications
- Enhances user experience with minimal resources

4. Explain zoom control in android.

Ans:- **Zoom control in android**

zoom control in android refers to the ui elements or mechanisms that allow users to zoom in (enlarge) or zoom out (shrink) the content on the screen. It is commonly used in applications that display images, maps, documents, or any view where users may need a closer or wider view. Android provides built-in support for zooming using specific ui components and gestures.

Purpose of zoom control

- To improve content visibility
- To allow detailed viewing of images or maps
- To provide user-friendly navigation
- To enhance accessibility
- To offer better control over content size

Ways to implement zoom control in android

1. Zoomcontrols widget
2. Pinch-to-zoom gesture (multi-touch)
3. Zoombuttonscontroller (deprecated)
4. Scalegesturedetector with custom views
5. Built-in zoom support in webview / mapview / imageview libraries
6. Third-party libraries (photoview, touchimageview, etc.)

Zoomcontrols widget (built-in ui component)

zoomcontrols is a simple widget that provides two buttons:

- **Zoom in (+)**
- **Zoom out (–)**

This widget is part of android.widget.zoomcontrols and is mainly used to control zoom manually in a view programmatically.

Xml example of zoomcontrols

```
<zoomcontrols
    android:id="@+id/zoomcontrols"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_centerInParent="true"/>
```

Using zoomcontrols in activity

```
Zoomcontrols zoom = findViewById(r.id.zoomcontrols);
```

```
Zoom.setOnZoomInClickListener(new View.OnClickListener() {
    @Override
    public void onClick(View v) {
        // perform zoom in action
    }
})
```

```
}  
});
```

```
Zoom.setOnZoomOutClickListener(new View.OnClickListener() {  
    @Override  
    public void onClick(View v) {  
        // perform zoom out action  
    }  
});
```

Pinch-to-zoom (gesture-based zooming)

modern android apps prefer pinch-to-zoom using `ScaleGestureDetector`.

- User uses two fingers to zoom in/out.
- Provides natural and intuitive zoom experience.

Built-in zoom support examples

- **WebView:** `webView.getSettings().setBuiltInZoomControls(true);`
- **MapView / google maps:** automatically supports pinch and double-tap zoom
- **ImageView (with libraries):** `photoview`, `glide`, etc. Provide zoom features

Advantages of zoom control

- Improves readability and accessibility
- Provides better user interaction
- Works with images, documents, maps, etc.
- Can be implemented via widget or gestures
- Enhances overall app experience